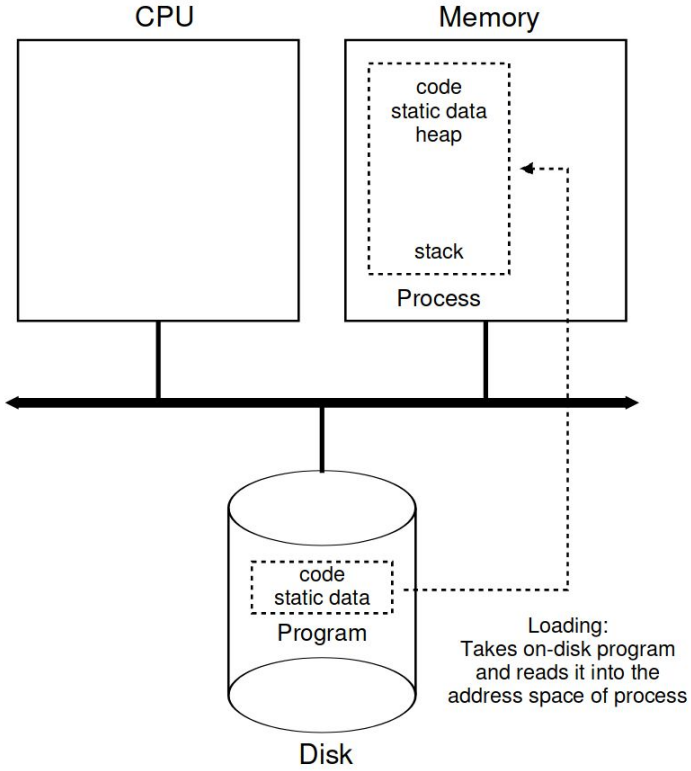
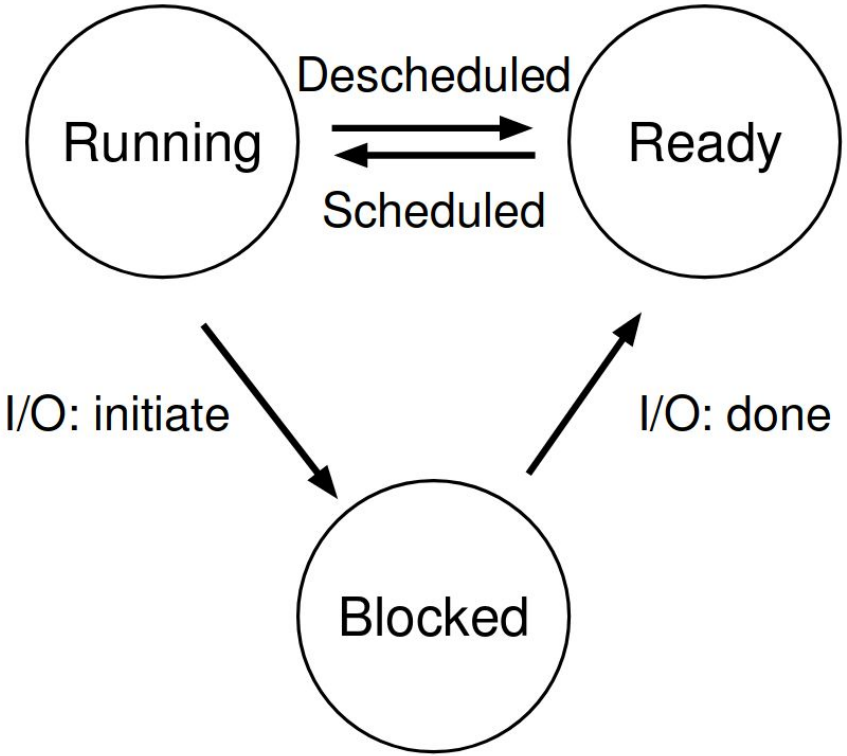
Абстракция: Процесс

Лекция №2

Загрузка: превращение программы в процесс



Процесс: промежуточные состояния



Состояние процессов (только CPU)

Time	Process_0	Process_1	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	
4	Running	Ready	Process_0 now done
5	–	Running	
6	–	Running	
7	–	Running	
8	–	Running	Process_1 now done

Состояние процессов (CPU и I/O)

Time	Process_0	Process_1	Notes
1	Running	Ready	
2	Running	Ready	
3	Running	Ready	Process_0 initiates I/O
4	Blocked	Running	Process_0 is blocked,
5	Blocked	Running	so Process_1 runs
6	Blocked	Running	
7	Ready	Running	I/O done
8	Ready	Running	Process_1 now done
9	Running	–	
10	Running	–	Process_0 now done

Process API

Лекция №2

Вызов fork()

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <unistd.h>
4
5   int main(int argc, char *argv[]) {
6       printf("hello world (pid:%d)\n", (int) getpid());
7       int rc = fork();
8       if (rc < 0) {          // fork failed; exit
9           fprintf(stderr, "fork failed\n");
10          exit(1);
11      } else if (rc == 0) { // child (new process)
12          printf("hello, I am child (pid:%d)\n", (int) getpid());
13      } else {               // parent goes down this path (main)
14          printf("hello, I am parent of %d (pid:%d)\n",
15                  rc, (int) getpid());
16      }
17      return 0;
18  }
19
```

Результат вызова fork()

```
prompt> ./p1
hello world (pid:29146)
hello, I am parent of 29147 (pid:29146)
hello, I am child (pid:29147)
prompt>
```

Результат вызова fork()

```
prompt> ./p1
hello world (pid:29146)
hello, I am child (pid:29147)
hello, I am parent of 29147 (pid:29146)
prompt>
```

Вызов fork() и wait()

```
1   #include <stdio.h>
2   #include <stdlib.h>
3   #include <unistd.h>
4   #include <sys/wait.h>
5
6   int main(int argc, char *argv[]) {
7       printf("hello world (pid:%d)\n", (int) getpid());
8       int rc = fork();
9       if (rc < 0) {          // fork failed; exit
10          fprintf(stderr, "fork failed\n");
11          exit(1);
12      } else if (rc == 0) { // child (new process)
13          printf("hello, I am child (pid:%d)\n", (int) getpid());
14      } else {               // parent goes down this path (main)
15          int rc_wait = wait(NULL);
16          printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
17                 rc, rc_wait, (int) getpid());
18      }
19      return 0;
20  }
21
```

Результат вызова fork() и wait()

```
prompt> ./p2
hello world (pid:29266)
hello, I am child (pid:29267)
hello, I am parent of 29267 (rc_wait:29267) (pid:29266)
prompt>
```

Вызов fork(), wait(), exec()

```
7   int main(int argc, char *argv[]) {
8       printf("hello world (pid:%d)\n", (int) getpid());
9       int rc = fork();
10      if (rc < 0) {          // fork failed; exit
11          fprintf(stderr, "fork failed\n");
12          exit(1);
13      } else if (rc == 0) { // child (new process)
14          printf("hello, I am child (pid:%d)\n", (int) getpid());
15          char *myargs[3];
16          myargs[0] = strdup("wc");   // program: "wc" (word count)
17          myargs[1] = strdup("p3.c"); // argument: file to count
18          myargs[2] = NULL;           // marks end of array
19          execvp(myargs[0], myargs);  // runs word count
20          printf("this shouldn't print out");
21      } else {                // parent goes down this path (main)
22          int rc_wait = wait(NULL);
23          printf("hello, I am parent of %d (rc_wait:%d) (pid:%d)\n",
24                 rc, rc_wait, (int) getpid());
25      }
26      return 0;
27  }
```

Результат вызова fork(), wait(), exec()

```
prompt> ./p3
hello world (pid:29383)
hello, I am child (pid:29384)
      29     107    1030 p3.c
hello, I am parent of 29384 (rc_wait:29384) (pid:29383)
prompt>
```

Пример программы с перенаправлением вывода

```
8   int main(int argc, char *argv[]) {
9       int rc = fork();
10      if (rc < 0) {           // fork failed; exit
11          fprintf(stderr, "fork failed\n");
12          exit(1);
13      } else if (rc == 0) { // child: redirect standard output to a file
14          close(STDOUT_FILENO);
15          open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
16
17          // now exec "wc"...
18          char *myargs[3];
19          myargs[0] = strdup("wc");   // program: "wc" (word count)
20          myargs[1] = strdup("p4.c"); // argument: file to count
21          myargs[2] = NULL;           // marks end of array
22          execvp(myargs[0], myargs);  // runs word count
23      } else {                // parent goes down this path (main)
24          int rc_wait = wait(NULL);
25      }
26      return 0;
27  }
```

Результат выполнения

```
prompt> ./p4
prompt> cat p4.output
      32     109     846 p4.c
prompt>
```